

SSGMCE	SHRI SANT GAJANAN MAHARAJ COLLEGE OF ENGG.		LABORATORY MANUAL
	PRACTICAL EXPERIMENT INSTRUCTION SHEET		
	EXPERIMENT TITLE : WRITE A PROGRAM TO PERFORM VARIOUS NATURAL LANGUAGE PROCESSING TASKS USING NLTK LIBRARY IN PYTHON		
EXPERIMENT NO. : SSGMCE/WI/IT/6IT09/08		ISSUE NO. : 00	ISSUE DATE : 06-01-2025
REV. DATE : NA	REV. NO. : 00	DEPTT. : INFORMATION TECHNOLOGY	
LABORATORY : COMPUTER SKILL LAB IV			SEMESTER : VI PAGE : 1 OF 4

01 AIM : Write a program to perform various natural language processing tasks using NLTK library in python

02 SCOPE : The experiment focuses on use of NLTK library in python to perform various natural language processing operations.

03 THEORY :

NLP:

NLP is a field of artificial intelligence that focuses on enabling computers to understand, interpret, and generate human language.

Natural Language Toolkit (NLTK)

It is one of the largest Python libraries for performing various Natural Language Processing tasks. From rudimentary tasks such as text pre-processing to tasks like vectorized representation of text – **NLTK's API** has covered everything. In this article, we will accustom ourselves to the basics of NLTK and perform some crucial NLP tasks: Tokenization, Stemming, Lemmatization, and POS Tagging.

To incorporate the usage of additional resources, such as recourses of languages other than English – you can run the following in a python script. It has to be done only once when you are running it for the first time in your system.

```
import nltk
nltk.download('all')
```

let's perform some basic operations on text data using NLTK.

Tokenization

Tokenization refers to break down the text into smaller units. It entails splitting paragraphs into sentences and sentences into words. It is one of the initial steps of any NLP pipeline. Let us have a look at the two major kinds of tokenization that NLTK provides:

Work Tokenization

It involves breaking down the text into words.

"I study Machine Learning on NPTEL." will be word-tokenized as
['I', 'study', 'Machine', 'Learning', 'on', 'NPTEL', '.'].

Sentence Tokenization

It involves breaking down the text into individual sentences.

Example:

"I study Machine Learning on NPTEL. Currently, I'm studying NLP"

will be sentence-tokenized as

['I study Machine Learning on NPTEL.', 'Currently, I'm studying NLP.']

In Python, both these tokenizations can be implemented in NLTK as follows:

Tokenization using NLTK

```
from nltk import word_tokenize, sent_tokenize
sent = "NPTEL is a great learning platform.\nIt is one of the best for Computer Science students."
print(word_tokenize(sent))
print(sent_tokenize(sent))
```

Output:

['NPTEL', 'is', 'a', 'great', 'learning', 'platform', '.', 'It', 'is', 'one', 'of', 'the', 'best', 'for', 'Computer', 'Science', 'students', '.']

['NPTEL is a great learning platform.', 'It is one of the best for Computer Science students.']

Stemming and Lemmatization

When working with Natural Language, we are not much interested in the form of words – rather, we are concerned with the meaning that the words intend to convey. Thus, we try to map every word of the language to its root/base form. This process is called canonicalization.

E.g. The words 'play', 'plays', 'played', and 'playing' convey the same action – hence, we can map them all to their base form i.e. 'play'.

Now, there are two widely used canonicalization techniques: Stemming and Lemmatization.

Stemming

Stemming generates the base word from the inflected word by removing the affixes of the word. It has a set of pre-defined rules that govern the dropping of these affixes. It must be noted that stemmers might not always result in semantically meaningful base words. Stemmers are faster and computationally less expensive than lemmatizers.

In the following code, we will be stemming words using Porter Stemmer – one of the most widely used stemmers:

```
from nltk.stem import PorterStemmer
```

```
# create an object of class PorterStemmer
porter = PorterStemmer()
print(porter.stem("play"))
print(porter.stem("playing"))
print(porter.stem("plays"))
print(porter.stem("played"))
```

Output:

Commun

The stemmer reduces the word 'communication' to a base word 'commun' which is meaningless in itself.

Lemmatization

Lemmatization involves grouping together the inflected forms of the same word. This way, we can reach out to the base form of any word which will be meaningful in nature. The base form here is called the Lemma.

Lemmatizers are slower and computationally more expensive than stemmers.

Example:

'play', 'plays', 'played', and 'playing' have 'play' as the lemma.

In Python, both these tokenizations can be implemented in NLTK as follows:

```
from nltk.stem import WordNetLemmatizer
# create an object of class WordNetLemmatizer
lemmatizer = WordNetLemmatizer()
print(lemmatizer.lemmatize("plays", 'v'))
print(lemmatizer.lemmatize("played", 'v'))
print(lemmatizer.lemmatize("play", 'v'))
print(lemmatizer.lemmatize("playing", 'v'))
```

Output:

play
play
play
play

Please note that in lemmatizers, we need to pass the Part of Speech of the word along with the word as a function argument.

Also, lemmatizers always result in meaningful base words. Let us take the same example as we took in the case for stemmers.

```
from nltk.stem import WordNetLemmatizer

# create an object of class WordNetLemmatizer
lemmatizer = WordNetLemmatizer()
print(lemmatizer.lemmatize("Communication", 'v'))
```

Output:

Communication

Part of Speech Tagging

Part of Speech (POS) tagging refers to assigning each word of a sentence to its part of speech. It is significant as it helps to give a better syntactic overview of a sentence.

Example:

"Python is a Computer Programming Language."
Let's see how NLTK's POS tagger will tag this sentence.

In Python, both these tokenizations can be implemented in NLTK as follows:

```
from nltk import pos_tag  
from nltk import word_tokenize
```

```
text = " Python is a computer promming language."  
tokenized_text = word_tokenize(text)  
tags = tokens_tag = pos_tag(tokenized_text)  
tags
```

Output:

```
[('Python', 'NNP'),  
 ('is', 'VBZ'),  
 ('a', 'DT'),  
 ('Computer', 'NNP'),  
 ('Programming', 'NNP'),  
 ('language', 'NN'),  
 ('.', '.')]
```

Conclusion

In conclusion, the Natural Language Toolkit (NLTK) works as a powerful Python library that a wide range of tools for Natural Language Processing (NLP). From fundamental tasks like text pre-processing to more advanced operations such as semantic reasoning, NLTK provides a versatile API that caters to the diverse needs of language-related tasks